WEB TECH

2 MARK QUESTIONS

1. What is internet and how does it work?
   Ans: The internet is a global network of interconnected computers and devices that communicate using standardized protocols to exchange data instantly across vast distances. It works by breaking information into packets, routing them through various networks, and reassembling them at the destination.

2. what are the main difference between webclint and web server?

Ans:

Feature	Web Client	Web Server
Role	Requests data from the server	Responds to client requests with data
Location	Runs on the user's device (e.g., PC, phone)	Runs on a remote server or data center
Examples	Web browsers (Chrome, Firefox), mobile apps	Apache, Nginx, Microsoft IIS
Function	Displays content, sends user input	Stores websites, processes requests
Initiates Communication	Yes (client starts the request)	No (server waits for requests)
User Interaction	Direct (user sees and interacts with it)	Indirect (user doesn't see it directly)
Technology Used	HTML, CSS, JavaScript	PHP, Python, Node.js, databases

3. what is css and why is it used in web development?
   Ans: **CSS (Cascading Style Sheets)** is a stylesheet language used in web development to control the presentation of HTML elements. It is used to define styles like colors, fonts, layouts, and spacing, making web pages visually appealing and consistent. CSS helps separate content from design, improves maintainability, and enables responsive design across different devices.

4. Explain the concept of css box model ?

Ans : **CSS Box Model** is a layout structure that defines how elements are displayed and spaced on a webpage. It consists of four parts:

1. **Content** – The actual text or image inside the element.
2. **Padding** – Space between the content and the border.
3. **Border** – A line surrounding the padding and content.
4. **Margin** – Space outside the border that separates the element from others.

This model helps control the size, spacing, and layout of elements in web design.

5. What is javascript and how it is used in web pages ?

Ans : **JavaScript** is a scripting language used to create dynamic and interactive content on web pages. It runs in the browser and allows developers to respond to user actions, validate forms, update content without reloading, and create animations. JavaScript enhances user experience by making web pages more responsive and functional.

6. What is the main purpose of using loops in javascript?

Ans : **The main purpose of using loops in JavaScript** is to execute a block of code repeatedly until a specified condition is met. Loops help reduce code repetition, making programs more efficient and easier to manage. They are commonly used for tasks like iterating over arrays, processing user input, or performing repeated calculations.

7. What do you mean by DHTML ? mention its main components?

Ans: **DHTML (Dynamic HTML)** is a collection of technologies used to create interactive and animated websites by combining HTML, CSS, JavaScript, and the Document Object Model (DOM). It allows web pages to change content and style dynamically without reloading.

Main components of DHTML:

1. **HTML** – Provides the basic structure of the webpage.
2. **CSS** – Styles the HTML elements.
3. **JavaScript** – Adds interactivity and dynamic behavior.
4. **DOM** – Represents the structure of the webpage and allows scripts to modify content and layout.

8. What are variables in php and how are they declared?

Ans: **Variables in PHP** are used to store data that can be reused and manipulated throughout a script. They act as containers for values like numbers, strings, or arrays.

Declaration in PHP:

- A variable starts with a dollar sign $ followed by the variable name.
- Example:
- $name = "John";
- $age = 25;

Variables are case-sensitive and do not require explicit data type declarations.

9. what are the different data type supported in php?

Ans : PHP supports several data types to store different kinds of values:

1. **Integer** – Whole numbers like 10, -5.
2. **Float (Double)** – Decimal numbers like 3.14, -0.99.
3. **String** – Text values like "Hello World".
4. **Boolean** – Represents true or false.
5. **Array** – Stores multiple values in a single variable.
6. **Object** – Represents instances of classes.
7. **NULL** – Represents a variable with no value.
8. **Resource** – Special type for external resources like file handles or database connections.

10. what is the use of array in php ?

Ans: **Use of Array in PHP (3 marks):**
An array in PHP is used to store multiple values in a single variable, making it easier to manage and organize data. It allows efficient handling of lists, such as user names, product items, or scores.

Types of arrays in PHP:

1. **Indexed Array** – Uses numeric keys to store values.
2. **Associative Array** – Uses named keys for easier reference.
3. **Multidimensional Array** – Stores arrays within arrays for complex data structures.

3 MARK QUESTIONS

1. Explain the client-server communication process on the internet?

Ans: The client-server communication process on the internet involves interaction between a client (e.g., a web browser) and a server (e.g., a web server) over a network, typically using the HTTP/HTTPS protocol.

Process:

1. **Request Initiation**: The client (e.g., a browser) sends a request to the server via a URL or user action (e.g., clicking a link).

2. **Request Transmission**: The request, formatted as an HTTP message, is sent over the internet to the server, specifying the desired resource (e.g., a webpage).

3. **Server Processing**: The server receives the request, processes it (e.g., retrieves a webpage or queries a database), and generates a response.

4. **Response Transmission**: The server sends an HTTP response back to the client, containing the requested resource (e.g., HTML, images) or an error message.

5. **Client Rendering**: The client receives the response, interprets it (e.g., renders HTML in the browser), and displays the content to the user.

Example: When a user types "www.example.com" in a browser, the browser sends an HTTP GET request to the server hosting the website. The server responds with the webpage's HTML, which the browser renders.

2. What are the different components of an HTTP request and response message?

HTTP Request Components:

1. **Request Line**: Contains the HTTP method (e.g., GET, POST), the resource path (e.g., /index.html), and the HTTP version (e.g., HTTP/1.1).

2. **Headers**: Metadata providing additional information, such as User-Agent, Accept (content type), and Host (server domain).

3. **Body** (optional): Contains data sent to the server, used in methods like POST (e.g., form data).

HTTP Response Components:

1. **Status Line**: Includes the HTTP version, status code (e.g., 200 OK, 404 Not Found), and status message.

2. **Headers**: Metadata like Content-Type (e.g., text/html), Content-Length, and Server information.

3. **Body** (optional): Contains the requested resource, such as HTML, JSON, or images.

Example:

- **Request**: `GET /index.html HTTP/1.1\nHost: www.example.com\nAccept: text/html`

- **Response**: `HTTP/1.1 200 OK\nContent-Type: text/html\n\n<html>...</html>`

3. What is the purpose of CSS in web development? Mention its advantages?

Purpose of CSS:

CSS (Cascading Style Sheets) is used to style and format the visual presentation of HTML elements on a webpage, separating content from design to enhance appearance and user experience.

Advantages:

1. **Separation of Concerns**: CSS separates styling from HTML structure, improving maintainability and readability.

2. **Consistency**: Enables uniform styling across multiple pages using a single CSS file.

3. **Flexibility**: Allows easy updates to the design without altering HTML.

4. **Enhanced User Experience**: Supports responsive design, animations, and visual effects for better interactivity.

5. **Faster Page Loading**: External CSS files can be cached, reducing load times.

Example: A CSS rule like `p { color: blue; font-size: 16px; }` styles all `<p>` elements with blue text and a 16px font size.

---

4. Explain the box model in CSS and describe its main properties?

The **CSS Box Model** represents the structure and layout of elements on a webpage, treating each element as a rectangular box.

Components of the Box Model:

1. **Content**: The actual content (text, images) with defined width and height.

2. **Padding**: The space between the content and the border, controlled by the `padding` property.

3. **Border**: A line surrounding the padding, defined by `border` (e.g., `border: 1px solid black;`).

4. **Margin**: The outer space outside the border, set by the `margin` property.

Main Properties:

- `width` and `height`: Set the content area's dimensions (e.g., `width: 200px;`).

- `padding`: Adds inner spacing (e.g., `padding: 10px;`).

- `border`: Defines the border style, width, and color (e.g., `border: 2px dashed red;`).

- `margin`: Sets outer spacing (e.g., `margin: 15px;`).

- `box-sizing`: Controls how width/height are calculated (e.g., `box-sizing: border-box;` includes padding and border in total dimensions).

Example: `div { width: 200px; padding: 10px; border: 1px solid black; margin: 20px; }` creates a box with specified content size, spacing, and borders.

---

5. What are variables and data types in JavaScript? Give examples?

Variables:

Variables in JavaScript are containers used to store data values, declared using `var`, `let`, or `const`.

Data Types:

1. **Primitive Types**:
   - **Number**: Represents integers or floats (e.g., `let num = 42;` or `let price = 19.99;`).
   - **String**: Text data enclosed in quotes (e.g., `let name = "Alice";`).
   - **Boolean**: True or false values (e.g., `let isActive = true;`).
   - **Undefined**: A variable declared but not assigned (e.g., `let x;`).
   - **Null**: Represents no value (e.g., `let data = null;`).
2. **Non-Primitive Types**:
   - **Object**: Stores key-value pairs (e.g., `let person = { name: "John", age: 30 };`).
   - **Array**: Ordered list of values (e.g., `let numbers = [1, 2, 3];`).

Examples:

```javascript
let age = 25; // Number
const greeting = "Hello"; // String
let isLoggedIn = false; // Boolean
let user = { id: 1, name: "Bob" }; // Object
```

---

6. Explain the role of the Document Object Model (DOM) in JavaScript?

The **Document Object Model (DOM)** is a programming interface that represents a webpage's structure as a tree of objects, allowing JavaScript to dynamically interact with and manipulate HTML and CSS.

Role of DOM in JavaScript:

1. **Accessing Elements**: JavaScript uses the DOM to select HTML elements (e.g., `document.getElementById("myId")`).

2. **Modifying Content**: Updates element content, attributes, or styles (e.g., `element.innerHTML = "New text";`).

3. **Event Handling**: Responds to user actions like clicks or inputs (e.g., `element.addEventListener("click", function);`).

4. **Dynamic Updates**: Creates, removes, or modifies elements dynamically (e.g., `document.createElement("div")`).

Example:

```javascript

// Change text of an element with id="demo"

document.getElementById("demo").innerHTML = "Hello, DOM!";

```

The DOM bridges JavaScript and the webpage, enabling interactive and dynamic web applications.

---

7. What is DHTML? How does it combine HTML, CSS, and JavaScript?

DHTML (Dynamic HTML):

DHTML refers to the combination of HTML, CSS, and JavaScript to create dynamic and interactive webpages that can change content or appearance without reloading.

How It Combines HTML, CSS, and JavaScript:

1. **HTML**: Provides the structure and content of the webpage (e.g., `<div id="box">Content</div>`).

2. **CSS**: Defines the presentation and styling of HTML elements (e.g., `#box { background: blue; }`).

3. **JavaScript**: Adds interactivity by manipulating the DOM and CSS (e.g., changing content or styles on user actions).

Example:

```html
<div id="box">Click me</div>
<style> #box { color: white; background: blue; padding: 10px; } </style>
<script>
  document.getElementById("box").onclick = function() {
    this.style.background = "red";
    this.innerHTML = "Clicked!";
  };
</script>
```

DHTML enables real-time updates, animations, and user-driven interactions by integrating these technologies.

---

8. What are constants and operators in PHP? Give examples?

Constants:

Constants in PHP are identifiers that store values that cannot be changed once defined. They are declared using `define()` or the `const` keyword.

Example:

```php
define("PI", 3.14); // Global constant
const MAX_USERS = 100; // Class or global constant
echo PI; // Outputs: 3.14
```

Operators:

Operators perform operations on variables/values. Types include:

1. **Arithmetic**: `+`, `-`, `*`, `/`, `%` (e.g., `$a = 10 + 5; // 15`).
2. **Comparison**: `==`, `===`, `!=`, `>`, `<` (e.g., `if ($a == $b)`).
3. **Logical**: `&&`, `||`, `!` (e.g., `if ($a > 0 && $b > 0)`).
4. **Assignment**: `=`, `+=`, `-=` (e.g., `$x += 5; // $x = $x + 5`).

Example:

```php
$a = 10; $b = 5;
$result = $a * $b; // Arithmetic: 50
if ($a > $b && $result > 0) { // Logical and Comparison
   echo "Valid";
}
```

---

9. Explain different types of loops in PHP with examples?

PHP supports several loop types for executing code repeatedly:

1. **For Loop**: Executes a block of code a specified number of times.

```php
for ($i = 1; $i <= 3; $i++) {
    echo "$i "; // Outputs: 1 2 3
}
```

2. **While Loop**: Executes as long as a condition is true.

```php
$i = 1;
while ($i <= 3) {
    echo "$i "; // Outputs: 1 2 3
    $i++;
}
```

3. **Do-While Loop**: Executes at least once, then continues if the condition is true.

```php
$i = 1;
do {
    echo "$i "; // Outputs: 1 2 3
    $i++;
} while ($i <= 3);
```

4. **Foreach Loop**: Iterates over arrays.

```php
$colors = ["red", "blue", "green"];
foreach ($colors as $color) {
    echo "$color "; // Outputs: red blue green  }
```

10. How can a PHP script be connected to a database? Mention the steps?

PHP can connect to a database (e.g., MySQL) using extensions like MySQLi or PDO.

Steps to Connect (Using MySQLi):

1. **Specify Database Credentials**: Define the hostname, username, password, and database name.

2. **Create a Connection**: Use `mysqli_connect()` to establish a connection to the database.

3. **Check Connection**: Verify if the connection was successful.

4. **Execute Queries**: Perform database operations (e.g., SELECT, INSERT).

5. **Close Connection**: Close the connection when done using `mysqli_close()`.

Example:

```
<?php
$host = "localhost";
$username = "root";
$password = "";
$dbname = "mydb";
// Create connection
$conn = mysqli_connect($host, $username, $password, $dbname);
// Check connection
if (!$conn) {
   die("Connection failed: " . mysqli_connect_error());
}

// Example query
$sql = "SELECT * FROM users";
$result = mysqli_query($conn, $sql);

// Close connection
mysqli_close($conn);
?>
```

6 MARK QUESTIONS

1. Explain in Detail the Client-Server Communication Model

The **client-server communication model** is a fundamental architecture for networked applications, particularly on the internet, where a client (e.g., a web browser) interacts with a server (e.g., a web server) to request and receive resources or services. This model is central to web communication and operates over protocols like HTTP/HTTPS.

Key Components:

1. **Client**: A device or software (e.g., browser, mobile app) that initiates a request for resources or services.

2. **Server**: A powerful system that hosts resources (e.g., webpages, databases) and responds to client requests.

3. **Network**: The medium (e.g., internet) that facilitates communication using protocols like TCP/IP.

4. **Protocol**: Rules governing communication, such as HTTP for web interactions or HTTPS for secure communication.

Detailed Process of Client-Server Communication:

1. **Request Initiation**:

   - The client, typically a user's browser, initiates a request when a user performs an action (e.g., enters a URL, clicks a link, or submits a form).

   - The request is formatted as an HTTP message, specifying the method (e.g., GET, POST), resource path, and protocol version.

2. **Domain Name Resolution**:

   - The client uses the Domain Name System (DNS) to resolve the server's domain name (e.g., www.example.com) to an IP address (e.g., 192.0.2.1).

   - This ensures the client knows where to send the request.

3. **Establishing Connection**:

- The client establishes a TCP connection with the server using the IP address and port (typically 80 for HTTP or 443 for HTTPS).
- A three-way handshake (SYN, SYN-ACK, ACK) ensures a reliable connection.

4. **Sending the Request**:
   - The client sends an HTTP request, which includes:
     - **Request Line**: E.g., `GET /index.html HTTP/1.1`.
     - **Headers**: Metadata like `Host: www.example.com`, `User-Agent`, `Accept`.
     - **Body** (optional): Data sent in POST requests (e.g., form data).
   - Example:
     ```
     GET /index.html HTTP/1.1
     Host: www.example.com
     Accept: text/html
     User-Agent: Mozilla/5.0
     ```

5. **Server Processing**:
   - The server receives the request and processes it:
     - Identifies the requested resource (e.g., a webpage, image, or API data).
     - May interact with a database or application logic to generate dynamic content.
     - Prepares an HTTP response based on the request.

6. **Sending the Response**:
   - The server sends an HTTP response, which includes:
     - **Status Line**: E.g., `HTTP/1.1 200 OK` or `HTTP/1.1 404 Not Found`.
     - **Headers**: Metadata like `Content-Type: text/html`, `Content-Length`.
     - **Body**: The requested resource (e.g., HTML, JSON, or image).
   - Example:
     ```
     HTTP/1.1 200 OK
     ```

Content-Type: text/html

Content-Length: 1234

<html><body><h1>Welcome</h1></body></html>

```

7. **Client Rendering**:

- The client receives the response and processes it (e.g., the browser renders HTML, CSS, and JavaScript).

- If additional resources (e.g., images, scripts) are needed, the client sends further requests.

8. **Connection Closure**:

- The connection may be closed (in HTTP/1.0) or kept alive (in HTTP/1.1 or HTTP/2) for subsequent requests.

#### **Key Features**:

- **Scalability**: Servers can handle multiple clients simultaneously.

- **Statelessness**: HTTP is stateless; each request is independent unless managed by cookies or sessions.

- **Reliability**: TCP ensures data is delivered accurately.

- **Security**: HTTPS encrypts communication to protect data.

#### **Advantages**:

- Centralized data management on the server.

- Supports diverse clients (browsers, apps).

- Scalable for large user bases.

#### **Disadvantages**:

- Server overload during high traffic.

- Dependency on network connectivity.

- Single point of failure if the server goes down.
```

Example Scenario:

When a user visits `www.example.com`, the browser (client) sends a GET request to the server. The server retrieves `index.html`, sends it back, and the browser renders the webpage. If the user submits a form, a POST request is sent with form data, and the server processes it (e.g., stores it in a database) before responding.

OR . Write a Detailed Note on the Structure of an HTML Document

An **HTML (HyperText Markup Language) document** is the backbone of a webpage, defining its structure and content using a standardized markup format. It consists of elements and tags that browsers interpret to render text, images, links, and other content.

Structure of an HTML Document:

An HTML document follows a hierarchical structure, encapsulated within specific tags. Below is a detailed breakdown of its components:

1. **<!DOCTYPE html>**:

   - Declares the document type as HTML5, informing browsers to use the latest HTML standard for rendering.

   - Placed at the top of the document.

   - Example: `<!DOCTYPE html>`

2. **<html>**:

   - The root element that encapsulates the entire HTML document.

   - Includes the `lang` attribute to specify the language (e.g., `lang="en"` for English).

   - Example: `<html lang="en">`

3. **<head>**:

   - Contains metadata about the document, which is not displayed on the webpage but is used by browsers and search engines.

   - Common elements within `<head>`:

     - `<title>`: Sets the webpage title displayed in the browser tab (e.g., `<title>My Website</title>`).

- `<meta>`: Provides metadata like character encoding (e.g., `<meta charset="UTF-8">`) or viewport settings for responsive design (e.g., `<meta name="viewport" content="width=device-width, initial-scale=1.0">`).

- `<link>`: Links external resources like CSS files (e.g., `<link rel="stylesheet" href="styles.css">`).

- `<script>`: Links or embeds JavaScript (e.g., `<script src="script.js"></script>`).

- Example:

```html
<head>
    <title>My Website</title>
    <meta charset="UTF-8">
    <meta name="description" content="A sample webpage">
    <link rel="stylesheet" href="styles.css">
</head>
```

4. **<body>**:

- Contains the visible content of the webpage, such as text, images, links, forms, and multimedia.

- Common elements within `<body>`:

- **Headings**: `<h1>` to `<h6>` for titles (e.g., `<h1>Welcome</h1>`).

- **Paragraphs**: `<p>` for text content (e.g., `<p>This is a paragraph.</p>`).

- **Links**: `<a>` for hyperlinks (e.g., `<a href="https://example.com">Visit Example</a>`).

- **Images**: `<img>` for images (e.g., `<img src="image.jpg" alt="Description">`).

- **Lists**: `<ul>` (unordered) or `<ol>` (ordered) for lists (e.g., `<ul><li>Item 1</li></ul>`).

- **Divisions**: `<div>` for grouping content (e.g., `<div class="container">Content</div>`).

- **Forms**: `<form>` for user input (e.g., `<form action="/submit" method="post">`).

- Example:

```html
<body>
    <h1>Welcome to My Website</h1>
    <p>This is a sample paragraph.</p>
    <a href="https://example.com">Click here</a>
    <img src="image.jpg" alt="Sample Image">
```

```
    </body>
    ```
```

#### **Complete HTML Document Example**:

```html
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>My Sample Webpage</title>
   <link rel="stylesheet" href="styles.css">
</head>
<body>
   <header>
      <h1>Welcome to My Website</h1>
   </header>
   <main>
      <p>This is the main content of the page.</p>
      <a href="https://example.com">Visit Example</a>
   </main>
   <footer>
      <p>© 2025 My Website</p>
   </footer>
</body>
</html>
```

Key Features of HTML Structure:

- **Hierarchical**: Elements are nested in a tree-like structure (DOM), with parent and child relationships.
```

- **Semantic Tags**: HTML5 introduced semantic elements like `<header>`, `<footer>`, `<article>`, and `<section>` to improve accessibility and SEO.

- **Extensibility**: HTML integrates with CSS for styling and JavaScript for interactivity.

- **Cross-Browser Compatibility**: Modern HTML5 is supported by all major browsers.

Importance of HTML Structure:

- **Content Organization**: Provides a clear structure for content, making it easy to style and manipulate.

- **Accessibility**: Semantic tags improve screen reader compatibility.

- **SEO**: Proper structure enhances search engine indexing.

- **Maintainability**: Well-structured HTML is easier to update and debug.

Best Practices:

- Always include `<!DOCTYPE html>` to ensure standards-compliant rendering.

- Use semantic tags for better structure and accessibility.

- Validate HTML using tools like the W3C Validator to ensure correctness.

- Keep the `<head>` minimal and relevant to metadata.

- Use lowercase tags and proper nesting for consistency.

2.What is CSS? Discuss Its Syntax, Properties, and Advantages

What is CSS?

CSS (Cascading Style Sheets) is a stylesheet language used to describe the presentation and visual styling of a document written in HTML or XML. It controls the layout, colors, fonts, and overall appearance of webpages, separating the content (handled by HTML) from the design. Introduced by the World Wide Web Consortium (W3C) in 1996, CSS enables developers to create responsive, consistent, and aesthetically pleasing websites. It "cascades" because styles can be inherited or overridden based on specificity, source order, and inheritance rules.

Syntax of CSS

CSS syntax consists of rulesets that target HTML elements and apply styles. A basic CSS rule includes:

- **Selector**: Specifies which HTML elements the style applies to (e.g., element names, classes, IDs, or attributes).

- **Declaration Block**: Enclosed in curly braces `{}` and contains one or more declarations.

- **Declaration**: A property-value pair separated by a colon `:`, ending with a semicolon `;`.

Basic Structure:

```css
selector {
   property: value;
   property: value;
}
```

Types of Selectors:

1. **Element Selector**: Targets all instances of an HTML tag (e.g., `p { color: blue; }` styles all `<p>` elements).

2. **Class Selector**: Targets elements with a specific class (e.g., `.highlight { background: yellow; }` for `<div class="highlight">`).

3. **ID Selector**: Targets a unique element by ID (e.g., `#header { font-size: 24px; }` for `<div id="header">`).

4. **Attribute Selector**: Targets elements with specific attributes (e.g., `input[type="text"] { border: 1px solid gray; }`).

5. **Pseudo-Class/Pseudo-Element**: For states or parts (e.g., `a:hover { color: red; }` for link hover, `p::first-line { font-weight: bold; }`).

6. **Combinators**: For relationships (e.g., `div p { margin: 10px; }` for `<p>` inside `<div>`, or `.class1 + .class2 { padding: 5px; }` for adjacent siblings).

Ways to Apply CSS:

- **Inline**: Directly in HTML (e.g., `<p style="color: red;">Text</p>`).
- **Internal**: In `<style>` tags within `<head>` (e.g., `<style> p { color: red; } </style>`).
- **External**: Linked via `<link rel="stylesheet" href="styles.css">`, promoting reusability.

Example CSS Code:

```css
/* Comment: This is a basic stylesheet */
body {
   font-family: Arial, sans-serif;  /* Property: font-family; Value: Arial, sans-serif */
   background-color: #f0f0f0;
}

h1 {
   color: #333;
   text-align: center;
}

.button {
   padding: 10px 20px;
   border-radius: 5px;
}
```

Properties in CSS

CSS properties define specific aspects of an element's style. There are hundreds, categorized as:

1. **Text Properties**: `color` (text color, e.g., `color: rgb(255, 0, 0);`), `font-size` (e.g., `font-size: 16px;`), `font-family`, `text-align`, `line-height`.

2. **Box Properties**: Related to layout (e.g., `width: 200px;`, `height: auto;`, `margin: 10px;`, `padding: 15px;`, `border: 1px solid black;`).

3. **Background Properties**: `background-color` (e.g., `background-color: white;`), `background-image` (e.g., `background-image: url('image.jpg');`), `background-repeat`.

4. **Positioning Properties**: `position` (e.g., `position: relative;`), `top`, `left`, `display` (e.g., `display: flex;`), `float`.

5. **Transition/Animation Properties**: `transition` (e.g., `transition: all 0.3s ease;`), `animation`.

6. **Flexbox/Grid Properties**: For layouts (e.g., `flex-direction: row;`, `grid-template-columns: repeat(3, 1fr);`).

Properties can use units like px, em, %, rem, vh/vw, or values like colors (hex, rgb, hsl), keywords (e.g., auto, none), or functions (e.g., calc(100% - 20px)).

Advantages of CSS

1. **Separation of Concerns**: CSS separates styling from HTML content, making code cleaner, easier to maintain, and reusable across pages.

2. **Consistency and Reusability**: A single CSS file can style an entire website, ensuring uniform design and reducing redundancy.

3. **Improved Performance**: External CSS files are cached by browsers, leading to faster load times for subsequent pages.

4. **Flexibility and Responsiveness**: Media queries (e.g., `@media (max-width: 600px) { body { font-size: 14px; } }`) enable responsive design for different devices.

5. **Enhanced User Experience**: Supports animations, transitions, and visual effects (e.g., hover effects), making websites more interactive.

6. **Accessibility and SEO Benefits**: Proper CSS improves readability (e.g., via contrast ratios) and helps search engines by focusing HTML on semantics.

7. **Cross-Browser Compatibility**: Modern CSS standards ensure consistent rendering across browsers like Chrome, Firefox, and Safari.

8. **Efficiency in Development**: Tools like CSS preprocessors (e.g., Sass) extend functionality, but core CSS is lightweight and easy to learn.

Disadvantages (Brief Note): Can lead to specificity conflicts in large projects; older browsers may not support advanced features.

OR. Explain the CSS Box Model in Details

The **CSS Box Model** is a core concept in CSS that defines how elements are rendered on a webpage. It treats every HTML element as a rectangular "box" with layers that determine its size, spacing, and positioning. This model is crucial for layout design, as it calculates the total space an element occupies, including content, padding, borders, and margins. Introduced in CSS1, it ensures consistent rendering across browsers.

Components of the CSS Box Model

The box model consists of four concentric layers, from innermost to outermost:

1. **Content Area**:

   - The innermost layer holding the actual content (e.g., text, images).

   - Dimensions are set by `width` and `height` properties (e.g., `width: 200px; height: 100px;`).

   - If not specified, it auto-adjusts based on content.

   - Overflow can be managed with `overflow` (e.g., `overflow: hidden;` to clip excess content).

2. **Padding**:

   - The transparent space between the content and the border.

   - Adds internal spacing without affecting the background.

   - Properties: `padding` (shorthand, e.g., `padding: 10px 20px 10px 20px;` for top/right/bottom/left), or individually (`padding-top: 10px;`).

   - Units: px, em, %, etc. Padding is always inside the box and included in the background color/image.

3. **Border**:

   - A visible or invisible line surrounding the padding and content.

   - Defines the edge of the box's "inner" area.

   - Properties: `border` (shorthand, e.g., `border: 2px solid black;`), or separately (`border-width: 2px; border-style: solid; border-color: black;`).

   - Styles: none, solid, dashed, dotted, double, etc. Borders can be rounded with `border-radius` (e.g., `border-radius: 10px;`).

4. **Margin**:

- The outermost transparent space outside the border, creating gaps between adjacent elements.

- Collapses vertically between elements (e.g., two 10px margins become 10px total).

- Properties: `margin` (shorthand, e.g., `margin: 20px auto;` for vertical 20px and horizontal centering), or individually (`margin-left: 15px;`).

- Negative margins are allowed for overlapping effects.

Visual Representation (Conceptual Diagram):

```
+---------------------------+  ← Margin (outer space)
|       Margin            |
|  +---------------------+  |
|  |      Border       |  |
|  |  +---------------+  |  |
|  |  |   Padding     |  |  |
|  |  |  +---------+  |  |  |
|  |  |  | Content |  |  |  ← Width/Height apply here
|  |  |  +---------+  |  |  |
|  |  +---------------+  |  |
|  +---------------------+  |
+---------------------------+
```

- **Total Width Calculation** (Standard Model): Total width = content width + left/right padding + left/right border + left/right margin.

- Example: For `width: 200px; padding: 10px; border: 5px solid; margin: 20px;`, total width = 200 + 20 (padding) + 10 (border) + 40 (margin) = 270px.

Box-Sizing Property

- Controls how `width` and `height` are calculated.

- **content-box** (default): Width/height apply only to content; padding/border add extra space.

- **border-box**: Width/height include content, padding, and border (margins are still outside). This simplifies layouts (e.g., `box-sizing: border-box;`).

- Example: With `border-box`, a `width: 200px;` element includes padding and border in the 200px total.

Key Properties and Behaviors

- **Display**: Affects box type (e.g., `display: block;` for full-width boxes, `display: inline;` for inline flow, `display: flex;` for flexible layouts).

- **Positioning Integration**: Box model interacts with `position` (static, relative, absolute, fixed) for layout control.

- **Inheritance and Specificity**: Styles cascade; more specific selectors override (e.g., ID > class > element).

- **Responsive Adjustments**: Use media queries to modify box properties for different screen sizes.

- **Browser Defaults**: Elements like `<div>` are block-level (full width), while `<span>` are inline.

Example CSS Code Demonstrating Box Model:

```css
.box {
   width: 200px;            /* Content width */
   height: 100px;           /* Content height */
   padding: 15px;           /* Inner spacing */
   border: 5px dashed red;    /* Border style */
   margin: 20px;            /* Outer spacing */
   box-sizing: border-box;    /* Includes padding/border in width/height */
   background-color: lightblue;
}
```

HTML Usage: `<div class="box">Content Here</div>`

Importance and Applications

- **Layout Control**: Essential for grids, flexbox, and responsive designs.

- **Common Issues**: Misunderstanding leads to overflow or unexpected spacing; use developer tools to inspect boxes.

- **Advantages**: Provides precise control over element sizing and spacing, enabling complex UIs.

- **Evolution**: CSS3 added features like `box-shadow` (e.g., `box-shadow: 2px 2px 5px gray;`) and `outline` for non-space-occupying borders.

3. What is JavaScript? Explain Its Variables, Operators, Functions, Loops, and Conditional Statements in Details

What is JavaScript?

JavaScript is a high-level, interpreted programming language primarily used for adding interactivity and dynamic behavior to webpages. Initially developed in 1995 by Brendan Eich at Netscape, it runs in web browsers (client-side) and, with environments like Node.js, on servers (server-side). JavaScript is a versatile, multi-paradigm language supporting object-oriented, functional, and imperative programming. It works seamlessly with HTML and CSS via the Document Object Model (DOM) to manipulate webpage content, respond to user actions, and handle asynchronous tasks (e.g., API calls).

Variables in JavaScript

Variables store data values and are declared using `var`, `let`, or `const`.

- **var**: Function-scoped, allows redeclaration and reassignment (older, less preferred).

- **let**: Block-scoped, allows reassignment but not redeclaration, ideal for changing values.

- **const**: Block-scoped, cannot be reassigned or redeclared, used for constant values (though object properties can change).

Data Types:

1. **Primitive**: Number (`42`, `3.14`), String (`"Hello"`), Boolean (`true`/`false`), Undefined (`let x;`), Null (`let x = null`), Symbol, BigInt.

2. **Non-Primitive**: Object (`{ key: value }`), Array (`[1, 2, 3]`), Function.

Example:

```javascript

var name = "Alice"; // String

let age = 25; // Number

const PI = 3.14; // Constant

let person = { id: 1, name: "Bob" }; // Object

```

Operators in JavaScript

Operators perform operations on variables/values. Categories include:

1. **Arithmetic**: `+` (addition), `-` (subtraction), `*` (multiplication), `/` (division), `%` (modulus), `++` (increment), `--` (decrement).

   - Example: `let sum = 5 + 3; // 8`

2. **Comparison**: `==` (loose equality), `===` (strict equality), `!=`, `!==`, `>`, `<`, `>=`, `<=`.

   - Example: `5 === "5"; // false (strict, checks type)`

3. **Logical**: `&&` (AND), `||` (OR), `!` (NOT).

   - Example: `if (age > 18 && age < 65) { ... }`

4. **Assignment**: `=`, `+=`, `-=`, `*=`, `/=`.

   - Example: `let x = 10; x += 5; // x = 15`

5. **Ternary**: Conditional operator (`condition ? valueIfTrue : valueIfFalse`).

   - Example: `let status = age >= 18 ? "Adult" : "Minor";`

6. **Bitwise** and **Type** operators: Less common but include `&`, `|`, `typeof`, `instanceof`.

Functions in JavaScript

Functions are reusable blocks of code that perform specific tasks, invoked by calling their name. They can take parameters and return values.

Types:

1. **Function Declaration**: `function add(a, b) { return a + b; }`

2. **Function Expression**: `const multiply = function(a, b) { return a * b; };`

3. **Arrow Function** (ES6): `const divide = (a, b) => a / b;`

4. **Anonymous Functions**: Functions without names, often used as callbacks.

Features:

- **Parameters**: Variables passed to functions (e.g., `a, b`).

- **Default Parameters**: `function greet(name = "Guest") { return "Hello, " + name; }`

- **Return**: Outputs a value (e.g., `return result;`).

- **Closures**: Functions retaining access to their outer scope’s variables.

Example:

```javascript

function calculateArea(width, height) {
```

```
  return width * height;
}
console.log(calculateArea(5, 3)); // Outputs: 15

const square = num => num * num;
console.log(square(4)); // Outputs: 16
```

Loops in JavaScript

Loops execute code repeatedly based on conditions. Types include:

1. **for Loop**: Known iteration count.

   ```javascript
   for (let i = 1; i <= 3; i++) {
     console.log(i); // Outputs: 1, 2, 3
   }
   ```

2. **while Loop**: Runs while a condition is true.

   ```javascript
   let i = 1;
   while (i <= 3) {
     console.log(i); // Outputs: 1, 2, 3
     i++;
   }
   ```

3. **do-while Loop**: Runs at least once, then checks condition.

   ```javascript
   let i = 1;
   do {
     console.log(i); // Outputs: 1, 2, 3
     i++;
   } while (i <= 3);
```

```
```

4. **for...of Loop**: Iterates over iterable objects (e.g., arrays).

```javascript
let colors = ["red", "blue", "green"];
for (let color of colors) {
    console.log(color); // Outputs: red, blue, green
}
```

5. **for...in Loop**: Iterates over object properties.

```javascript
let person = { name: "Alice", age: 25 };
for (let key in person) {
    console.log(key, person[key]); // Outputs: name Alice, age 25
}
```

Conditional Statements in JavaScript

Conditional statements control code execution based on conditions.

1. **if Statement**: Executes code if a condition is true.

```javascript
let age = 20;
if (age >= 18) {
    console.log("Adult"); // Outputs: Adult
}
```

2. **if-else Statement**: Provides an alternative if the condition is false.

```javascript
if (age >= 18) {
    console.log("Adult");
} else {
    console.log("Minor");
```

```
}
```

3. **else-if Statement**: Handles multiple conditions.

```javascript
let score = 85;
if (score >= 90) {
    console.log("A");
} else if (score >= 80) {
    console.log("B"); // Outputs: B
} else {
    console.log("C");
}
```

4. **switch Statement**: Handles multiple cases for a single expression.

```javascript
let day = 2;
switch (day) {
    case 1:
        console.log("Monday");
        break;
    case 2:
        console.log("Tuesday"); // Outputs: Tuesday
        break;
    default:
        console.log("Invalid");
}
```

5. **Ternary Operator**: Compact conditional (see Operators section).

Advantages of JavaScript

- Adds interactivity (e.g., form validation, animations).

- Runs client-side, reducing server load.

- Integrates with DOM for dynamic updates.

- Supports modern frameworks (React, Vue) and server-side (Node.js).

Disadvantages

- Browser compatibility issues for older versions.

- Security risks (e.g., XSS) if not handled properly.

- Performance can be slower for complex computations.

OR. Explain the Document Object Model (DOM) in JavaScript

The **Document Object Model (DOM)** is a programming interface provided by web browsers that represents the structure of an HTML or XML document as a tree of objects. It allows JavaScript to dynamically access, manipulate, and update the content, structure, and style of a webpage without reloading. Standardized by the W3C, the DOM bridges HTML/CSS and JavaScript, enabling interactive web applications.

Structure of the DOM

- **Tree Representation**: The DOM organizes a webpage as a hierarchical tree, where each HTML element, attribute, and text is a **node**.

  - **Element Nodes**: HTML tags (e.g., `<div>`, `<p>`).

  - **Text Nodes**: Text content within elements (e.g., "Hello" in `<p>Hello</p>`).

  - **Attribute Nodes**: Attributes like `id` or `class` (e.g., `id="main"`).

- **Root Node**: The `<html>` element, with `<head>` and `<body>` as children.

- **Parent-Child Relationships**: Elements like `<div>` can contain child elements (e.g., `<p>`).

- Example DOM Tree for:

  ```html
  <!DOCTYPE html>
  <html>
    <body>
      <div id="main">
        <p>Hello</p>
  ```

```
    </div>
  </body>
</html>
```

DOM Tree:

```
Document
  └── html
      ├── head
      └── body
          └── div (id="main")
              └── p
                  └── text ("Hello")
```

Key Features of the DOM

1. **Dynamic Manipulation**: JavaScript can change content, attributes, or styles in real-time.

2. **Event Handling**: Responds to user actions (e.g., clicks, keypresses).

3. **Cross-Browser Compatibility**: Standardized to work across modern browsers.

4. **Object-Based**: Each node is an object with properties (e.g., `innerHTML`, `className`) and methods (e.g., `appendChild()`).

Core DOM Operations

1. **Accessing Elements**:

   - `document.getElementById("id")`: Selects an element by ID.

   - `document.getElementsByClassName("class")`: Returns a collection of elements by class.

   - `document.querySelector(".class")`: Selects the first element matching a CSS selector.

   - `document.querySelectorAll("p")`: Returns all matching elements.

   - Example: `let elem = document.getElementById("main");`

2. **Modifying Content**:

- `innerHTML`: Gets/sets HTML content (e.g., `elem.innerHTML = "<p>New Content</p>";`).
- `textContent`: Gets/sets text content (e.g., `elem.textContent = "New Text";`).
- Example: `document.querySelector("p").textContent = "Updated";`

3. **Modifying Attributes**:
   - `getAttribute("name")`: Retrieves an attribute value.
   - `setAttribute("name", "value")`: Sets an attribute.
   - Example: `elem.setAttribute("class", "highlight");`

4. **Modifying Styles**:
   - `style.property`: Updates CSS (e.g., `elem.style.color = "red";`).
   - Example: `document.querySelector(".box").style.backgroundColor = "blue";`

5. **Creating/Removing Elements**:
   - `document.createElement("tag")`: Creates a new element.
   - `appendChild()`, `removeChild()`: Adds/removes elements from the DOM.
   - Example:
     ```javascript
     let newDiv = document.createElement("div");
     newDiv.textContent = "New Div";
     document.body.appendChild(newDiv);
     ```

6. **Event Handling**:
   - `addEventListener(event, function)`: Attaches event listeners for user actions.
   - Example:
     ```javascript
     document.getElementById("btn").addEventListener("click", () => {
       alert("Button clicked!");
     });
     ```

Example: DOM Manipulation

```html
<div id="main">
   <p class="text">Hello, World!</p>
   <button onclick="updateContent()">Update</button>
</div>
<script>
function updateContent() {
   let para = document.querySelector(".text");
   para.textContent = "Updated Content";
   para.style.color = "green";
   let newP = document.createElement("p");
   newP.textContent = "New Paragraph";
   document.getElementById("main").appendChild(newP);
}
</script>
```

This code updates text, changes style, and adds a new paragraph when the button is clicked.

Advantages of the DOM

- Enables dynamic, interactive webpages (e.g., real-time form validation).
- Supports event-driven programming for user interactions.
- Integrates with CSS for styling and JavaScript for logic.
- Allows modular updates without full page reloads (e.g., via AJAX).

Disadvantages

- Performance overhead for large DOM trees or frequent manipulations.
- Security risks (e.g., XSS if `innerHTML` is misused).
- Browser inconsistencies in older versions.

Best Practices

- Use `querySelector` for specific selections but avoid overuse for performance.

- Cache DOM queries (e.g., store `document.getElementById("id")` in a variable).

- Prefer `textContent` over `innerHTML` for security.

- Use modern frameworks (e.g., React) for complex DOM operations to optimize performance.

The DOM is the cornerstone of interactive web development, enabling JavaScript to transform static HTML into dynamic applications.

4. What is PHP? Explain Variables, Data Types, Operators, Loops, and Decision-Making Statements in PHP with Proper Examples

What is PHP?

PHP (Hypertext Preprocessor) is a widely used, open-source, server-side scripting language designed for web development. Originally created by Rasmus Lerdorf in 1994, it is embedded within HTML and executes on the server, generating dynamic content (e.g., webpages, forms, databases) before sending it to the client's browser. PHP is platform-independent, supports object-oriented programming (OOP), and integrates seamlessly with databases like MySQL. It powers popular CMS like WordPress and frameworks like Laravel. PHP scripts are saved with a `.php` extension and run via a web server (e.g., Apache) with a PHP interpreter.

Variables in PHP

Variables in PHP store data and are declared with a dollar sign `$` followed by the variable name (case-sensitive, starts with a letter or underscore). No explicit type declaration is needed; PHP is loosely typed and determines types dynamically. Variables can be reassigned.

Scope: Global (outside functions), Local (inside functions), Static (retains value across function calls).

Example:

```php
<?php
$name = "Alice"; // String variable
$age = 25; // Integer variable
$price = 19.99; // Float variable
```

```
echo $name . " is " . $age . " years old."; // Outputs: Alice is 25 years old.
?>
```

Data Types in PHP

PHP supports scalar, compound, and special data types. Types are not strictly enforced, but functions like `gettype()` can check them.

1. **Scalar Types**:
   - **Integer**: Whole numbers (e.g., `42`, `-5`).
   - **Float** (Double): Decimal numbers (e.g., `3.14`).
   - **String**: Text in single/double quotes (e.g., `"Hello"`, `'World'`).
   - **Boolean**: `true` or `false`.

2. **Compound Types**:
   - **Array**: Indexed or associative collections (e.g., `array(1, 2, 3)` or `["key" => "value"]`).
   - **Object**: Instances of classes (e.g., `new MyClass()`).

3. **Special Types**:
   - **Resource**: External resources (e.g., database connections).
   - **NULL**: No value (e.g., `$var = null;`).

Example:

```php
<?php
$intVar = 100; // Integer
$floatVar = 10.5; // Float
$stringVar = "PHP"; // String
$boolVar = true; // Boolean
$arrayVar = array("apple", "banana"); // Array
$nullVar = null; // NULL
```

```
echo gettype($intVar); // Outputs: integer
echo "<br>";
print_r($arrayVar); // Outputs: Array ( [0] => apple [1] => banana )
?>
```

Operators in PHP

Operators perform operations on variables/values. Categories include:

1. **Arithmetic**: `+`, `-`, `*`, `/`, `%` (modulus), `**` (exponentiation).
   - Example: `$a = 10; $b = 3; $sum = $a + $b; // 13`

2. **Assignment**: `=`, `+=`, `-=`, `*=`, `/=`, `%=`.
   - Example: `$x = 5; $x += 2; // $x = 7`

3. **Comparison**: `==` (equal), `===` (identical, checks type), `!=`, `!==`, `>`, `<`, `>=`, `<=`.
   - Example: `5 == "5"; // true (loose)`, `5 === "5"; // false (strict)`

4. **Logical**: `&&` (AND), `||` (OR), `!` (NOT).
   - Example: `if ($a > 0 && $b > 0) { echo "Positive"; }`

5. **Increment/Decrement**: `++`, `--`.
   - Example: `$i = 1; $i++; // $i = 2`

6. **String**: `.` (concatenation).
   - Example: `"Hello" . " World"; // "Hello World"`

Example Code:

```php
<?php
```

```
$a = 10; $b = 5;
echo $a * $b; // Outputs: 50 (Arithmetic)
echo "<br>";
if ($a > $b || $b == 0) { echo "Condition met"; } // Outputs: Condition met (Logical/Comparison)
?>
```

Loops in PHP

Loops execute code repeatedly. Types include:

1. **for Loop**: Known iterations.

   ```php
   <?php
   for ($i = 1; $i <= 3; $i++) {
       echo $i . " "; // Outputs: 1 2 3
   }
   ?>
   ```

2. **while Loop**: Condition-checked before execution.

   ```php
   <?php
   $i = 1;
   while ($i <= 3) {
       echo $i . " "; // Outputs: 1 2 3
       $i++;
   }
   ?>
   ```

3. **do-while Loop**: Executes at least once, condition-checked after.

```php
<?php
$i = 1;
do {
    echo $i . " "; // Outputs: 1 2 3
    $i++;
} while ($i <= 3);
?>
```

4. **foreach Loop**: Iterates over arrays.

```php
<?php
$fruits = ["apple", "banana", "cherry"];
foreach ($fruits as $fruit) {
    echo $fruit . " "; // Outputs: apple banana cherry
}
?>
```

Decision-Making Statements in PHP

These control program flow based on conditions.

1. **if Statement**: Executes if condition is true.

```php
<?php
$age = 20;
if ($age >= 18) {
    echo "Adult"; // Outputs: Adult
}
?>
```

```

2. **if-else Statement**: Alternative for false condition.

```php
<?php
if ($age >= 18) {
    echo "Adult";
} else {
    echo "Minor";
}
?>
```

3. **if-elseif-else Statement**: Multiple conditions.

```php
<?php
$score = 85;
if ($score >= 90) {
    echo "A";
} elseif ($score >= 80) {
    echo "B"; // Outputs: B
} else {
    echo "C";
}
?>
```

4. **switch Statement**: Multi-case selection.

```php
<?php
$day = 2;
```
```

```
switch ($day) {
    case 1:
        echo "Monday";
        break;
    case 2:
        echo "Tuesday"; // Outputs: Tuesday
        break;
    default:
        echo "Invalid";
}
?>
```

In summary, PHP's variables, data types, operators, loops, and decision-making statements enable efficient server-side scripting for dynamic web applications.

OR. Explain How PHP Can Be Connected to a Database? Mention the Steps with Examples

PHP can connect to databases like MySQL, PostgreSQL, or SQLite using extensions such as MySQLi (MySQL Improved) or PDO (PHP Data Objects). MySQLi is procedural/object-oriented and MySQL-specific, while PDO is more flexible, supporting multiple databases and prepared statements for security. Connections allow CRUD (Create, Read, Update, Delete) operations. Always use secure practices (e.g., avoid plain-text credentials in production).

Steps to Connect PHP to a Database (Using MySQLi for MySQL)

1. **Prepare Database Credentials**: Define the server hostname (e.g., "localhost"), username (e.g., "root"), password, and database name.

2. **Establish the Connection**: Use `mysqli_connect()` (procedural) or `new mysqli()` (OOP) to create a connection object.

3. **Check the Connection**: Verify if the connection succeeded; if not, handle errors (e.g., using `mysqli_connect_error()`).

4. **Execute Queries**: Perform SQL operations (e.g., SELECT, INSERT) using `mysqli_query()`.

5. **Process Results** (if applicable): Fetch data with `mysqli_fetch_assoc()` or similar for SELECT queries.

6. **Close the Connection**: Use `mysqli_close()` to free resources.

Example: Connecting to MySQL Database with MySQLi

Assume a MySQL database named "mydb" with a "users" table (columns: id, name, email).

```php
<?php
// Step 1: Define credentials
$servername = "localhost";
$username = "root";
$password = ""; // Use a secure password in production
$dbname = "mydb";

// Step 2: Create connection (OOP style)
$conn = new mysqli($servername, $username, $password, $dbname);

// Step 3: Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
echo "Connected successfully!<br>";

// Step 4: Execute a query (e.g., SELECT)
$sql = "SELECT id, name, email FROM users";
```

```
$result = $conn->query($sql);

// Step 5: Process results
if ($result->num_rows > 0) {
   while ($row = $result->fetch_assoc()) {
      echo "ID: " . $row["id"] . " - Name: " . $row["name"] . " - Email: " . $row["email"] . "<br>";
   }
} else {
   echo "0 results";
}

// Step 6: Close connection
$conn->close();
?>
```

Alternative: Using PDO

PDO is recommended for better security (e.g., prepared statements prevent SQL injection).

Steps (Similar to Above):

1. Define credentials.
2. Create PDO object with DSN (Data Source Name).
3. Set error mode (e.g., exceptions).
4. Execute queries.
5. Close (PDO closes automatically, but set to null if needed).

Example:

```php
<?php
$servername = "localhost";
$username = "root";
$password = "";
$dbname = "mydb";
```

```
try {
   // Step 2: Create PDO connection
   $conn = new PDO("mysql:host=$servername;dbname=$dbname", $username, $password);
   // Step 3: Set error mode
   $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
   echo "Connected successfully!<br>";

   // Step 4-5: Prepare and execute query
   $stmt = $conn->prepare("SELECT id, name, email FROM users");
   $stmt->execute();
   $result = $stmt->fetchAll(PDO::FETCH_ASSOC);

   foreach ($result as $row) {
      echo "ID: " . $row["id"] . " - Name: " . $row["name"] . " - Email: " . $row["email"] . "<br>";
   }
} catch(PDOException $e) {
   echo "Connection failed: " . $e->getMessage();
}

// Step 6: Close connection
$conn = null;
?>
```

Best Practices:

- Use prepared statements to prevent SQL injection.
- Store credentials securely (e.g., in environment variables or config files outside web root).
- Handle errors gracefully in production.
- For large applications, use ORM libraries like Doctrine.

This process enables PHP to interact with databases for dynamic data-driven websites.